

Sentinel: Technical Documentation

Deterministic AutoML + MLOps for predictive maintenance. Prototype Phase submission, ABB Accelerator, Theme 1.

1. Problem and approach

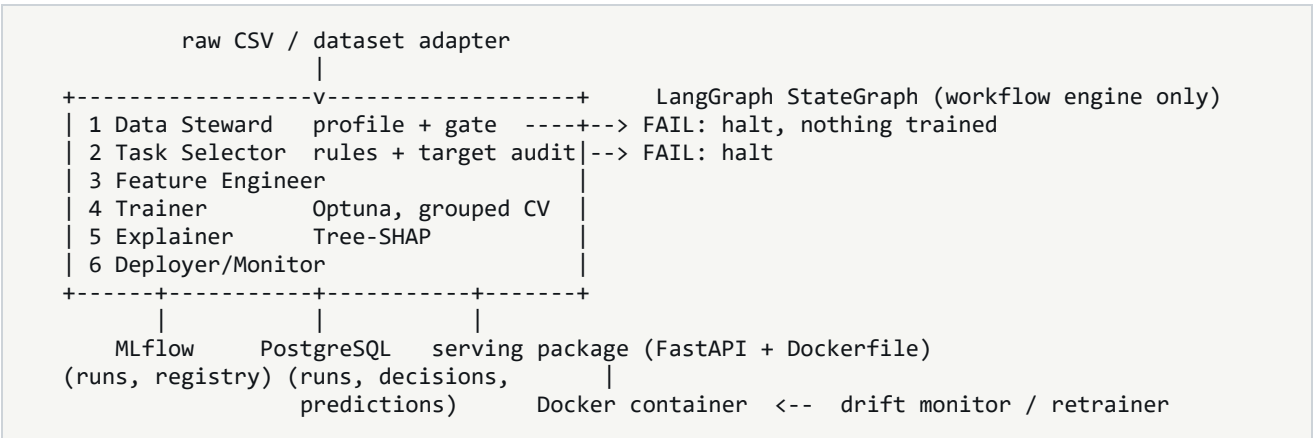
Predictive maintenance tools fail in a specific way: nobody can explain why the model flagged, or missed, a failure, so nobody acts on it. Wrapping the workflow in a generative model makes this worse, because its reasoning cannot be reproduced or audited.

Sentinel automates the path from raw sensor data to a deployed, explainable failure-prediction model, and makes one design commitment: **every decision in the pipeline is a fixed rule, a statistical test, or a cross-validated metric. No language model is called anywhere.** Machine learning is used where it is the right tool, for the failure-prediction models themselves.

The consequences are engineering properties, not slogans:

- **Reproducible.** Same data, same thresholds, same seed give the same decision log, the same leaderboard and the same run hash.
- **Auditable.** Each decision records the rule that fired, the numbers it read, and the threshold it compared against.
- **Explainable by construction.** Explanations are Tree-SHAP values turned into sentences by fixed templates.

2. Architecture



Component	File	Responsibility
Config	sentinel/config.py	Every threshold and the seed, in one place. Part of the run hash.

Component	File	Responsibility
Structure detection	<code>sentinel/structure.py</code>	Finds the entity column and the time counter from the data.
Data Steward	<code>sentinel/steward.py</code>	Scorecard and quality gate (two stages).
Selector	<code>sentinel/selector.py</code>	Task framing, target derivation, ranking metric, holdout split.
Features	<code>sentinel/features.py</code>	Serialisable feature spec and transforms, shared with the serving container.
Trainer	<code>sentinel/trainer.py</code>	Seeded Optuna search, cross-validated leaderboard.
Explainer	<code>sentinel/explainer.py</code>	SHAP contributions, ranked drivers, sentence templates, reliability table.
Deployer	<code>sentinel/deployer.py</code>	Serving package, in-process parity test, container smoke test.
Monitor / lifecycle	<code>sentinel/monitor.py</code> , <code>sentinel/lifecycle.py</code>	PSI/KS drift, retrain rule, champion/challenger.
Registry / store	<code>sentinel/registry.py</code> , <code>sentinel/store.py</code>	MLflow tracking and registry; SQLAlchemy persistence.
Graph	<code>sentinel/graph.py</code>	The six nodes and the conditional gate edges.
API + UI	<code>api/main.py</code> , <code>web/</code>	REST, server-sent events, single-page UI.

3. The decision rules

Each row is a rule the pipeline applies, with its threshold from `config.py` . Every firing is written to the decision ledger with its inputs.

Rule	Reads	Threshold	Outcome
Sample size	row count	warn < 1000, fail < 100	fail halts the run
Missing values	max per-column missing fraction	warn > 5%, fail > 30%	fail halts the run
Duplicates	duplicate-row share	warn > 1%	reported
Constant columns	unique values per column	1	column dropped
Identifier columns	one unique value per row, or > 50 categories	n/a	column dropped
Multicollinearity	pairwise r, VIF	$r \geq 0.95$, $VIF > 10$	reported (trees tolerate it)
Distribution shift	two-sample KS, early vs late life (or first vs second half)	$D \geq 0.2$, $p < 0.01$	reported; expected for degradation data
Structure	contiguous entity blocks with a +1 time counter	99% of steps	run-to-failure framing

Rule	Reads	Threshold	Outcome
Alarm horizon	median life	0.15 x median life	target = fails within horizon
Target type	distinct target values	binary, or > 10 continuous	classification or regression
Class balance	minority share	warn < 10%, fail < 0.5%	fail halts the run
Ranking metric	minority share	< 10%	average precision, else ROC-AUC (RMSE for regression)
Class weighting	minority share	< 30%	balanced weights on every candidate
Leakage guard	P(target=1 given flag=1) for each 0/1 column	>= 98%, support >= 10	column dropped
Champion	cross-validated metric	rank 1	selected; holdout never consulted
Alert threshold	out-of-fold scores	maximises F1	fixed and stored with the model
Drift	PSI of top-10 important features, early-life window	watch 0.10, retrain 0.25	retrain rule
Promotion	challenger vs incumbent on unseen units	strictly better	promote or keep

4. Reproducibility engineering

- **Seeds everywhere:** `SEED = 42` for Optuna's TPE sampler, LightGBM/XGBoost/scikit-learn estimators, fold splitting and holdout splitting.
- **Fixed parallelism:** `N_JOBS = 4` is a constant, not `os.cpu_count()`, and LightGBM runs with `deterministic=True, force_row_wise=True`.
- **Stable ordering:** series are stable-sorted by (entity, time); tie-breaks in the leaderboard are alphabetical.
- **The run hash** is the SHA-256 of a canonical JSON of: a fingerprint of the input frame, all options and thresholds, the full decision log, and the leaderboard (family, parameters, CV scores). Wall-clock times are excluded on purpose. Floats are rounded to 6 places.
- **Tested:** `test_pipeline_is_deterministic` runs the full pipeline twice in separate directories and asserts identical hashes, identical leaderboards and identical decision logs. On NASA C-MAPSS two independent full runs produced the same hash.

5. Feature engineering

For temporal data, each non-constant sensor becomes five features per time step, computed per unit over trailing windows (the start of a series is padded with its first value so training and serving agree): the current value, a 5-step mean, a 5-step standard deviation, a 10-step least-squares slope, and a 5-step change. The time counter is kept as a feature. For C-MAPSS FD001 that is 17 sensors x 5 + 1 = 86 features. For tabular data numeric columns pass through and categoricals are one-hot encoded with a category list frozen at training time.

Feature names are sanitised (brackets, quotes and commas break several model libraries) and mapped back to the original column names for display.

6. Training and evaluation

Candidates are LightGBM, XGBoost, Random Forest and a linear baseline (logistic or ridge). Each gets a seeded Optuna TPE search (6/6/3/3 trials in the default "fast" budget, 20/20/8/6 in "full"), scored by 5-fold cross-validation grouped by unit, so no unit is in both training and validation. The primary metric is ROC-AUC, average precision (rare failures) or RMSE. The alert threshold is the out-of-fold score that maximises F1. After ranking, every family is refit on all training data and scored on the holdout for transparency, but the holdout never influences selection.

7. Explainability

Contributions come from Tree-SHAP. For LightGBM and XGBoost the library's native `pred_contrib` is used (it is the same algorithm and needs no extra dependency in the serving image); `parity_check` proves it equals `shap.TreeExplainer` on 400 sampled rows (maximum absolute difference 0.0 on the C-MAPSS run).

For a prediction, the top drivers are ranked by absolute contribution. Each carries a z-score against the fleet baseline and its share of the explained signal. The sentence is assembled from a fixed template:

Engine 76 ALERT at cycle 205: failure risk >99%. Ps30 static pressure at HPC outlet, 5-cycle average above baseline (+1.9 sd), 15% of the signal; ...

Confidence is measured, not self-reported: the out-of-fold hit rate of the score band the prediction falls in ("historically 94% of scores in this band were followed by failure within the alarm horizon, n=2635").

8. Deployment

`deployer.package` writes a self-contained folder per model version:

```
model.(txt|json|joblib)  feature_spec.json  reference.json  metadata.json
sentinel/features.py      sentinel/explainer.py  serve.py
Dockerfile  requirements.txt  README.md
```

The service exposes `GET /health`, `GET /metadata`, and `POST /predict` (raw sensor rows in, risk, level, confidence and ranked drivers out). Because `features.py` and `explainer.py` are copied verbatim from training, training and serving cannot disagree about feature computation.

Two automated checks run on every pipeline run and the second on demand:

1. **Parity test:** the generated service is imported exactly as the container would import it and its scores are compared with the trained model's (`SERVING_PARITY` , maximum difference 0.0).
2. **Container smoke test:** builds the image, starts it, calls `/health` and `/predict` , compares scores (Docker socket required; on by default in `docker compose`).

Models are logged to MLflow (parameters, metrics, decision log, scorecard, leaderboard, feature spec as artifacts) and registered under `sentinel-<dataset>` with the alias `champion` .

9. Monitoring and retraining

The monitor stores quantile-bin reference distributions of every feature. Live data is compared with the training distribution using the Population Stability Index and a KS test. To avoid mistaking degradation itself for drift, the comparison uses the healthy early-life window (first 50 time steps). The retrain rule is fixed: any of the ten most important features at $PSI \geq 0.25$.

When the rule trips, `lifecycle.retrain` refits the champion's model family (same parameters) on training data plus half of the drifted units and scores incumbent and challenger on the other half. The challenger is promoted, registered as the next MLflow model version and packaged as a new service folder only if it is strictly better.

The UI's drift scenario injects a calibration offset (a number of training standard deviations) into one sensor. That injection is simulated; detection, retraining and promotion are real.

10. HTTP API

Method and path	Purpose
<code>GET /api/mode</code> , <code>GET /api/datasets</code>	Capabilities and available datasets
<code>POST /api/upload</code>	Upload a CSV
<code>POST /api/runs</code>	Start a run <code>{dataset, budget, task, target, horizon}</code>
<code>GET /api/runs</code> , <code>GET /api/runs/{id}</code> , <code>GET /api/runs/{id}/trace</code>	List runs, fetch a run, fetch its events
<code>GET /api/runs/{id}/events</code>	Server-sent events while the run executes
<code>POST /api/runs/{id}/predict</code>	Score raw rows through the generated service; logged to PostgreSQL
<code>POST /api/runs/{id}/deploy</code>	Container build and smoke test
<code>POST /api/runs/{id}/drift</code> , <code>POST /api/runs/{id}/retrain</code>	Drift scenario and champion/challenger
<code>GET /api/runs/{id}/lineage</code>	MLflow versions

11. Setup and testing

```
python scripts/fetch_data.py          # NASA C-MAPSS FD001 and UCI AI4I 2020 into ./data
docker compose up --build             # app :8000, MLflow :5000, PostgreSQL
python -m sentinel.cli run cmapss_fd001 --budget fast    # or run headless
pytest -q                             # 19 tests on synthetic data
```

Test coverage: structure detection, steward checks and the failing gate, leakage guard, the closed-form horizon rule, feature ordering stability and window correctness against pandas, PSI behaviour, end-to-end determinism, champion selection, serving parity, SHAP parity, the registry, the regression path, and a static import audit that fails if any language-model client appears in the package.

12. Scalability and feasibility

- **Data:** feature computation is vectorised per unit; training scales with rows x features x trials and the trial budget is a parameter. The same code path has been run on C-MAPSS (20,631 rows, 86 features) and AI4I (8,000 training rows).
- **Storage:** PostgreSQL for runs, decisions and predictions; MLflow server with a proxied artifact store for models. Both are standard, horizontally deployable services.
- **Serving:** each model version is an independent stateless container with one small dependency set, so it can be replicated behind any load balancer and rolled back by tag.
- **Adoption:** nothing leaves the machine. There are no external API calls at run time, which matters for plant networks. The decision ledger and run hash give reviewers something concrete to sign off.

13. Limitations and roadmap

- The hosted demo is a recorded real run; live training and uploads require the Docker stack.
- Survival analysis for censored data is not implemented; run-to-failure logs use classification or RUL regression.
- C-MAPSS FD001 is simulated with one operating condition. Next: FD002/FD004 (multiple operating conditions) and real plant data.
- Drift injection is simulated; a production deployment would feed the monitor from live sensor streams.
- Multiclass targets are rejected with a clear message rather than guessed at.

14. Data sources and licences

NASA C-MAPSS turbofan degradation data, NASA Prognostics Center of Excellence (public). UCI AI4I 2020 Predictive Maintenance Dataset (UCI ML Repository, CC BY 4.0). Sentinel's code is MIT licensed.